4.3 파라체인 배포 및 운영 프로세스 가이드

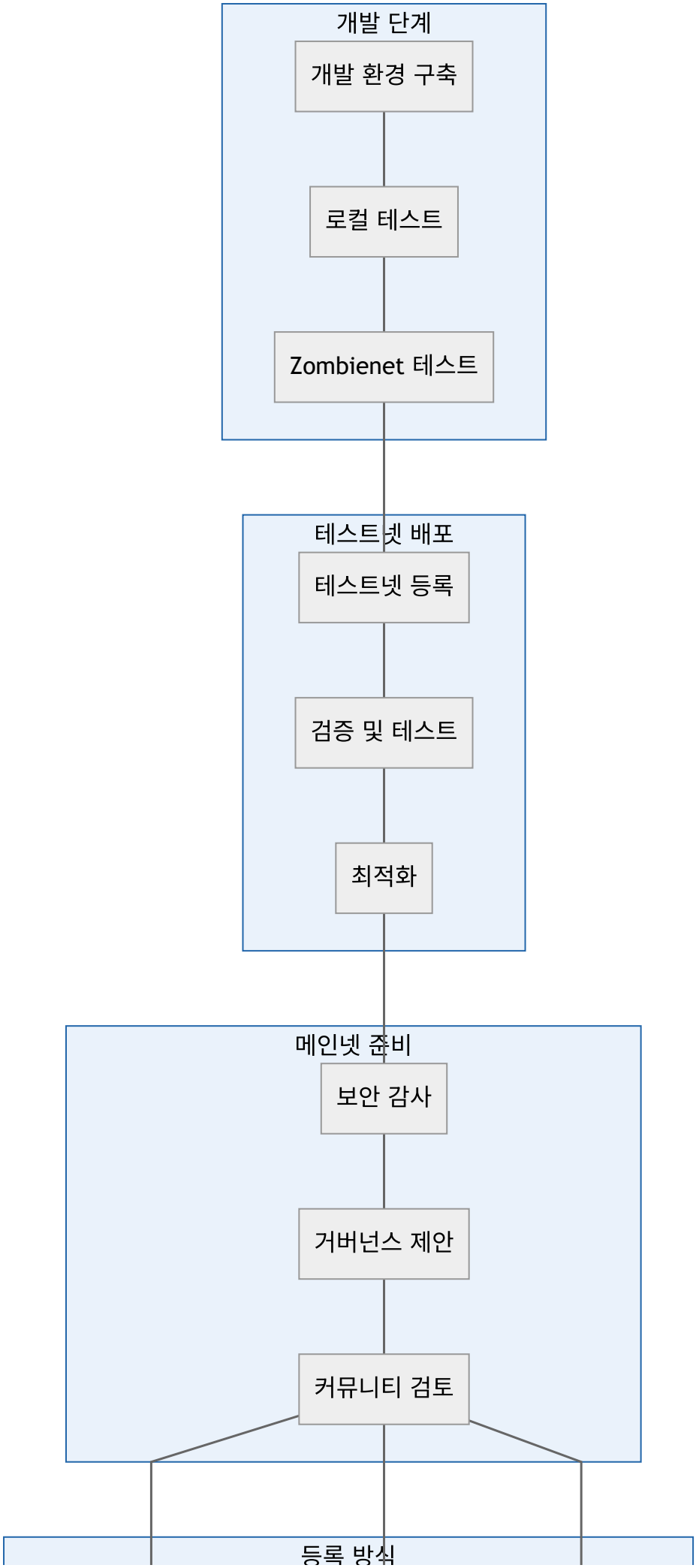
개요

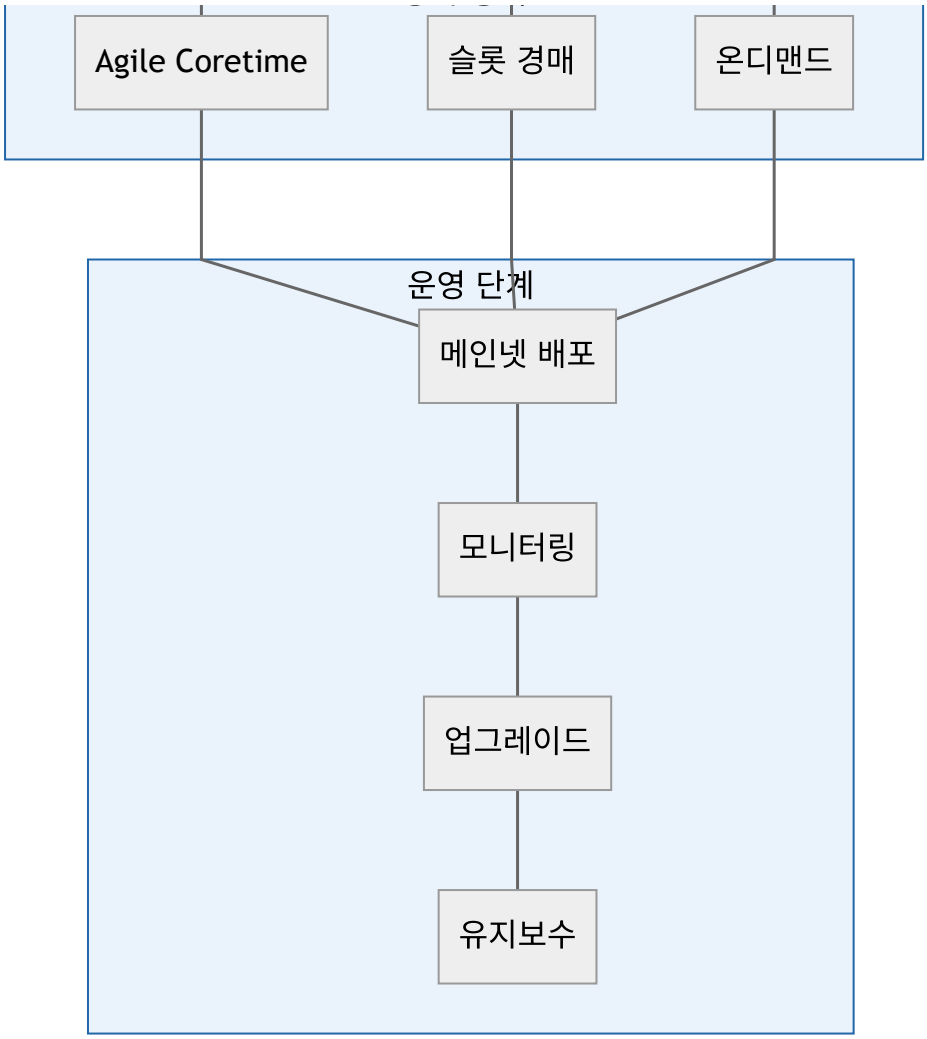
파라체인 배포는 단순한 블록체인 실행을 넘어서 Polkadot 릴레이 체인 생태계에 통합되는 복잡한 과정입니다. 이 과정은 개발, 테스트, 등록, 배포, 운영의 여러 단계를 거치며, 각 단계마다 특별한 고려사항과 절차가 필요합니다.

현재 Polkadot은 전통적인 슬롯 경매 시스템에서 Agile Coretime 모델로 전환하고 있어, 두 가지 방식 모두를 이해하는 것이 중요합니다.

파라체인 등록 과정

4.3.1 파라체인 생명주기


개발 단계
개발 환경 구축
로컬 테스트
Zombienet 테스트
테스트넷 배포
테스트넷 등록
검증 및 테스트
최적화
메인넷 준비
보안 감사
거버넌스 제안
커뮤니티 검토
등록 방식



4.3.2 파라체인 상태 관리

파라체인은 다음과 같은 상태를 가집니다:

```
// 파라체인 생명주기 상태
pub enum ParaLifecycle {
Onboarding, // 온보딩 중
Parathread, // 파라스레드 (온디맨드)
Parachain, // 파라체인 (전용 슬롯)
UpgradingParathread, // 파라스레드로 업그레이드 중
UpgradingParachain, // 파라체인으로 업그레이드 중
OffboardingParathread, // 파라스레드 오프보딩 중
```

```
OffboardingParachain, // 파라체인 오프보딩 중
}

// 파라체인 정보 구조
pub struct ParaGenesisArgs {
pub genesis_head: HeadData, // 제네시스 헤드 데이터
pub validation_code: ValidationCode, // 검증 코드 (WASM)
pub para_kind: ParaKind, // 파라체인 종류
}

pub enum ParaKind {
Parathread, // 온디맨드 파라체인
Parachain, // 전용 슬롯 파라체인
}
```

Agile Coretime 시스템

4.3.3 Coretime 개념

Agile Coretime은 Polkadot의 새로운 코어 할당 시스템으로, 더 유연하고 효율적인 자원 관리를 제공합니다.

핵심 개념

```
// 코어 할당 구조
```

```
pub struct CoreAssignment {
pub core: CoreIndex, // 코어 인덱스
pub para_id: ParaId, // 파라체인 ID
pub kind: AssignmentKind, // 할당 종류
pub group_idx: GroupIndex, // 검증자 그룹 인덱스
}

pub enum AssignmentKind {
Parachain, // 전용 파라체인
Parathread(CollatorId, u32), // 온디맨드 (콜레이터, 재시도 횟수)
}

// 코어 시간 스케줄
pub struct Schedule {
pub assignments: Vec<(CoreAssignment, PartsOf57600)>, // 할당과 시간 비율
pub end_hint: Option<BlockNumber>, // 종료 힌트
}
```

Coretime 구매 과정

```
// 1. 코어 예약
pub fn reserve_core(
origin: OriginFor<T>,
schedule: Schedule,
```

```
) -> DispatchResult {
// 코어 예약 로직
let core_count = Self::core_count();
ensure!(schedule.assignments.len() <= core_count as usize, Error::<T>::TooManyC
// 스케줄 검증
Self::validate_schedule(&schedule)?;
// 예약 실행
Self::do_reserve_core(schedule)?;
Ok(())
}


// 2. 리스 설정 (장기 할당)
pub fn set_lease(
origin: OriginFor<T>,
task: TaskId,
until: Timeslice,
) -> DispatchResult {
ensure_root(origin)?;
// 리스 설정
Leases::<T>::insert(&task, until);
// 코어타임 체인에 알림
let message = Xcm(vec![
Transact {
```

```
origin_kind: OriginKind::Superuser,

require_weight_at_most: T::WeightInfo::set_lease(),

call: BrokerRuntimePallets::Broker(CoretimeCalls::SetLease(task, until))

.encode()

.into(),

},

]);

T::XcmSender::send_xcm(T::CoretimeChain::get(), message)

.map_err(|_| Error::<T>::XcmSendFailed)?;

Ok(())

}
```

4.3.4 Coretime 기반 파라체인 등록

1. 코어 구매

```
# Coretime 체인에서 코어 구매

polkadot-js-api  tx.broker.purchase(

regionId,  //  지역  ID

price  //  가격

)


# 또는 2차 시장에서 구매

polkadot-js-api  tx.broker.transfer(
```

```
regionId,  //  지역  ID
newOwner  //  새  소유자
)
```

2. 파라체인 할당

```
# 구매한 코어를 파라체인에 할당
polkadot-js-api  tx.broker.assign(
regionId,  //  지역  ID
task,  //  태스크 (파라체인 ID)
finality  //  최종성  요구사항
)
```

3. 파라체인 등록

```
// 파라체인 등록 extrinsic
pub  fn  register_para(
origin: OriginFor<T>,
id: ParaId,
genesis_head: HeadData,
validation_code: ValidationCode,
) -> DispatchResult {
ensure_root(origin)?;
// 파라체인 ID 검증
```

```
ensure!(!Paras::<T>::contains_key(&id), Error::<T>::ParaAlreadyExists);

// 검증 코드 사전 검사 시작

let code_hash = validation_code.hash();

Self::schedule_code_check(code_hash, validation_code.clone())?;

// 파라체인 정보 저장

let para_info = ParaGenesisArgs {

genesis_head,

validation_code,

para_kind: ParaKind::Parachain,

};

PendingParas::<T>::insert(&id, para_info);

Self::deposit_event(Event::ParaRegistered(id));

Ok(())

}
```

전통적인 슬롯 경매 시스템

4.3.5 슬롯 경매 과정

전통적인 방식에서는 파라체인 슬롯을 경매를 통해 획득합니다.

경매 단계

```
// 경매 정보 구조

pub struct AuctionInfo<BlockNumber> {
```

```
pub lease_period_index: LeasePeriodIndex, // 리스 기간 인덱스
pub ending_period: BlockNumber, // 종료 기간
}

// 입찰 정보
pub struct BidInfo<AccountId, Balance> {
pub bidder: AccountId, // 입찰자
pub amount: Balance, // 입찰 금액
pub first_slot: LeasePeriodIndex, // 첫 번째 슬롯
pub last_slot: LeasePeriodIndex, // 마지막 슬롯
}
```

크라우드론 (Crowdloan)

```
// 크라우드론 정보
pub struct FundInfo<AccountId, Balance, BlockNumber> {
pub depositor: AccountId, // 예치자
pub verifier: Option<MultiSigner>, // 검증자
pub deposit: Balance, // 예치금
pub raised: Balance, // 모금액
pub end: BlockNumber, // 종료 블록
pub cap: Balance, // 목표 금액
pub last_contribution: LastContribution<BlockNumber>, // 마지막 기여
```

```
pub first_period: LeasePeriodIndex, // 첫 번째 기간
pub last_period: LeasePeriodIndex, // 마지막 기간
pub trie_index: TrieIndex, // 트라이 인덱스
}


// 크라우드론 생성
pub fn create_crowdloan(
origin: OriginFor<T>,
index: ParaId,
cap: BalanceOf<T>,
first_period: LeasePeriodIndex,
last_period: LeasePeriodIndex,
end: T::BlockNumber,
verifier: Option<MultiSigner>,
) -> DispatchResult {
let depositor = ensure_signed(origin)?;
// 파라체인 존재 확인
ensure!(paras::Pallet::<T>::is_valid_para(index), Error::<T>::InvalidParaId);
// 크라우드론 정보 생성
let fund_info = FundInfo {
depositor: depositor.clone(),
verifier,
deposit: T::SubmissionDeposit::get(),
```

```
raised: Zero::zero(),

end,

cap,

last_contribution: LastContribution::Never,

first_period,

last_period,

trie_index: Self::next_trie_index()?,

};

// 예치금 예약

T::Currency::reserve(&depositor, T::SubmissionDeposit::get())?;

// 크라우드론 등록

Funds::<T>::insert(&index, &fund_info);

Self::deposit_event(Event::Created(index));

Ok(())

}
```

테스트넷 배포 및 검증 과정

4.3.6 로컬 개발 환경 설정

Zombienet을 이용한 로컬 네트워크

```
# zombienet-config.toml

[relaychain]
```

```
default_command = "polkadot"

chain = "rococo-local"


[[relaychain.nodes]]

name = "alice"

validator = true

ws_port = 9944

rpc_port = 9933


[[relaychain.nodes]]

name = "bob"

validator = true

ws_port = 9955

rpc_port = 9934


[[relaychain.nodes]]

name = "charlie"

validator = true

ws_port = 9966

rpc_port = 9935


[[relaychain.nodes]]

name = "dave"
```

```
validator = true

ws_port = 9977

rpc_port = 9936



# 파라체인 설정

[[parachains]]

id = 1000

chain_spec_path = "./chain-spec.json"



[parachains.collator]

name = "collator-1000"

ws_port = 9988

rpc_port = 9987

command = "parachain-template-node"

args = ["--", "--execution=wasm"]



[[parachains]]

id = 2000

chain_spec_path = "./chain-spec-2000.json"



[parachains.collator]

name = "collator-2000"

ws_port = 9999
```

```
rpc_port = 9998

command = "parachain-template-node"

args = ["--", "--execution=wasm"]
```

네트워크 실행

```
# 1. 바이너리 빌드

cargo build --release -p polkadot

cargo build --release -p parachain-template-node


# 2. 체인 스펙 생성

./target/release/polkadot build-spec --chain rococo-local --disable-default-


# 3. 파라체인 체인 스펙 생성

./target/release/parachain-template-node build-spec --chain local --disable-


# 4. Zombienet 실행

zombienet --provider native spawn zombienet-config.toml
```

4.3.7 테스트넷 등록 과정

Rococo 테스트넷 등록

```
# 1. 제네시스 상태 및 WASM 내보내기
```

```
./target/release/parachain-template-node export-genesis-state --chain chain-s
./target/release/parachain-template-node export-genesis-wasm --chain chain-sp

# 2. 파라체인 등록 (sudo 권한 필요)
polkadot-js-api tx.sudo.sudo(
api.tx.paras.forceRegisterPara(
1000, // Para ID
genesisState, // Genesis state
genesisWasm // Genesis WASM
)
)

# 3. 슬롯 할당 (테스트넷에서는 sudo로 직접 할당 가능)
polkadot-js-api tx.sudo.sudo(
api.tx.assignedSlots.assignTempParachainSlot(
1000, // Para ID
SlotLeasePeriodStart.Immediate, // 시작 시점
tempSlotLeasePeriodLength, // 기간
null // 반복 없음
)
)
```

콜레이터 실행

```
# 콜레이터 노드 실행
./target/release/parachain-template-node \
--collator \
--alice \
--force-authoring \
--chain  chain-spec.json \
--base-path /tmp/parachain/alice \
--port  40333 \
--rpc-port 8844 \
-- \
--execution wasm \
--chain  rococo-local.json \
--port 30343 \
--rpc-port  9977
```

4.3.8 검증 및 테스트 절차

1. 기본 기능 검증

```
// 파라체인 연결 확인
const  api  =  await  ApiPromise.create({
provider:  new  WsProvider('ws://localhost:9988')
});
```

```
// 블록 생성 확인
const unsubscribe = await api.rpc.chain.subscribeNewHeads((header) => {
console.log(`Chain is at block: #${header.number}`);
});

// 파라체인 정보 확인
const parachainInfo = await api.query.parachainInfo.parachainId();
console.log(`Parachain ID: ${parachainInfo.toString()}`);
```

2. XCM 기능 테스트

```
// 크로스체인 자산 전송 테스트
const transferCall = api.tx.polkadotXcm.limitedReserveTransferAssets(
{ V3: { parents: 1, interior: { X1: { Parachain: 2000 } } } }, // 목적
{ V3: { parents: 0, interior: { X1: { AccountId32: { network: null, id: r
{ V3: [{ id: { Concrete: { parents: 0, interior: 'Here' } }, fun: { Fung
0, // 수수료 자산 인덱스
{ Limited: { refTime: 5000000000, proofSize: 65536 } } // 가중치 제한
);

await transferCall.signAndSend(alice);
```

3. 성능 테스트

```
# 블록 생성 시간 측정
polkadot-js-api query.system.number --ws ws://localhost:9988

# 트랜잭션 처리량 테스트
# 여러 트랜잭션을 동시에 전송하여 TPS 측정

# 메모리 및 CPU 사용량 모니터링
top -p $(pgrep parachain-template-node)
```

파라체인 업그레이드 및 거버넌스 연동

4.3.9 런타임 업그레이드 과정

파라체인 런타임 업그레이드는 릴레이 체인의 승인을 받아야 합니다.

업그레이드 프로세스

```
// 런타임 업그레이드 스케줄링
pub fn schedule_code_upgrade(
origin: OriginFor<T>,
new_code: ValidationCode,
) -> DispatchResult {
ensure_root(origin)?;
let para_id = parachain_info::Pallet::<T>::parachain_id();
```

```
let current_block = <frame_system::Pallet<T>>::block_number();

// 업그레이드 블록 계산 (최소 대기 시간 적용)

let upgrade_block = current_block + T::MinUpgradeDelay::get();

// 코드 해시 계산

let code_hash = new_code.hash();

// 업그레이드 스케줄링

PendingValidationCode::<T>::insert(&para_id, (upgrade_block, new_code));

// 릴레이 체인에 알림

let message = Xcm(vec![

Transact {

origin_kind: OriginKind::Superuser,

require_weight_at_most: Weight::from_parts(1_000_000_000, 64 * 1024),

call: RuntimeCall::Paras(paras::Call::schedule_code_upgrade {

para: para_id,

new_code: new_code.clone(),

}).encode().into(),

},

]);

T::XcmSender::send_xcm(Location::parent(), message)

.map_err(|_| Error::<T>::XcmSendFailed)?;

Self::deposit_event(Event::ValidationCodeUpgradeScheduled(para_id, upgrade_block

Ok(())
```

```
}
```

자동 업그레이드 시스템

```
// 블록 초기화 시 업그레이드 확인
impl<T: Config> Hooks<BlockNumberFor<T>> for Pallet<T> {
fn on_initialize(n: BlockNumberFor<T>) -> Weight {
let para_id = parachain_info::Pallet::<T>::parachain_id();
if let Some((upgrade_block, new_code)) = PendingValidationCode::<T>::get(&para_
if n >= upgrade_block {
// 업그레이드 실행
Self::apply_code_upgrade(para_id, new_code);
PendingValidationCode::<T>::remove(&para_id);
return T::DbWeight::get().reads_writes(2, 1);
}
}
T::DbWeight::get().reads(1)
}
}
```

4.3.10 거버넌스 통합

파라체인 거버넌스 설정

```
// 거버넌스 구성

impl pallet_democracy::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type Currency = Balances;

type EnactmentPeriod = EnactmentPeriod;

type LaunchPeriod = LaunchPeriod;

type VotingPeriod = VotingPeriod;

type VoteLockingPeriod = VoteLockingPeriod;

type MinimumDeposit = MinimumDeposit;

type ExternalOrigin = EnsureRootOrHalfCouncil;

type ExternalMajorityOrigin = EnsureRootOrHalfCouncil;

type ExternalDefaultOrigin = EnsureRoot<AccountId>;

type SubmitOrigin = EnsureSigned<AccountId>;

type FastTrackOrigin = EnsureRootOrTwoThirdsCouncil;

type InstantOrigin = EnsureRoot<AccountId>;

type InstantAllowed = InstantAllowed;

type FastTrackVotingPeriod = FastTrackVotingPeriod;

type CancellationOrigin = EnsureRoot<AccountId>;

type BlacklistOrigin = EnsureRoot<AccountId>;

type CancelProposalOrigin = EnsureRoot<AccountId>;

type VetoOrigin = pallet_collective::EnsureMember<AccountId, CouncilCollective>;

type CooloffPeriod = CooloffPeriod;
```

```
type Slash = Treasury;

type Scheduler = Scheduler;

type PalletsOrigin = OriginCaller;

type MaxVotes = MaxVotes;

type WeightInfo = pallet_democracy::weights::SubstrateWeight<Runtime>;

type MaxProposals = MaxProposals;

type Preimages = Preimage;

type MaxDeposits = MaxDeposits;

type MaxBlacklisted = MaxBlacklisted;

}
```

릴레이 체인 거버넌스와의 연동

```
// 릴레이 체인 거버넌스 제안을 통한 파라체인 관리

pub fn propose_parachain_upgrade(

origin: OriginFor<T>,

para_id: ParaId,

new_code: ValidationCode,

) -> DispatchResult {

let proposer = ensure_signed(origin)?;

// 제안 생성

let proposal = RuntimeCall::Paras(paras::Call::schedule_code_upgrade {

para: para_id,

new_code,
```

```
});

// 민주주의 팔렛을 통한 제안 제출

pallet_democracy::Pallet::<T>::propose(

origin,

proposal.into(),

T::MinimumDeposit::get(),

)

}
```

파라체인 모니터링 및 성능 최적화

4.3.11 모니터링 시스템 구축

Prometheus 메트릭 설정

```
// 커스텀 메트릭 정의

use prometheus::{Counter, Histogram, Registry};


pub struct ParachainMetrics {

pub blocks_produced: Counter,

pub block_production_time: Histogram,

pub xcm_messages_sent: Counter,

pub xcm_messages_received: Counter,

pub collation_time: Histogram,
```

```
}

impl ParachainMetrics {
pub  fn  new(registry: &Registry) -> Result<Self, prometheus::Error> {
Ok(Self {
blocks_produced: Counter::new(
"parachain_blocks_produced_total",
"Total number of blocks produced by this parachain"
)?,
block_production_time: Histogram::with_opts(
prometheus::HistogramOpts::new(
"parachain_block_production_seconds",
"Time taken to produce a block"
)
)?,
xcm_messages_sent: Counter::new(
"parachain_xcm_messages_sent_total",
"Total number of XCM messages sent"
)?,
xcm_messages_received: Counter::new(
"parachain_xcm_messages_received_total",
"Total number of XCM messages received"
)?,
```

```
collation_time: Histogram::with_opts(
prometheus::HistogramOpts::new(
"parachain_collation_seconds",
"Time taken to create a collation"
)
)?,
})
}
}
```

Grafana 대시보드 설정

```
# grafana-dashboard.json (일부)
{
"dashboard": {
"title": "Parachain Monitoring",
"panels": [
{
"title": "Block Production Rate",
"type": "graph",
"targets": [
{
"expr": "rate(parachain_blocks_produced_total[5m])",
```

```
"legendFormat": "Blocks/sec"
}
]
},
{
"title": "XCM Message Flow",
"type": "graph",
"targets": [
{
"expr": "rate(parachain_xcm_messages_sent_total[5m])",
"legendFormat": "Sent"
},
{
"expr": "rate(parachain_xcm_messages_received_total[5m])",
"legendFormat": "Received"
}
]
}
]
}
}
```

4.3.12 성능 최적화 전략

1. 블록 생성 최적화

```
// 블록 생성 시간 최적화
impl<T: Config> Pallet<T> {
fn optimize_block_production() {
// 트랜잭션 풀 관리
let pending_txs = TransactionPool::ready();
let optimal_batch_size = Self::calculate_optimal_batch_size();
// 배치 크기 조정
if pending_txs.len() > optimal_batch_size {
// 우선순위 기반 트랜잭션 선택
let selected_txs = Self::select_high_priority_transactions(
pending_txs,
optimal_batch_size
);
}
}
fn calculate_optimal_batch_size() -> usize {
// 네트워크 상태와 하드웨어 성능을 고려한 최적 배치 크기 계산
let network_latency = Self::get_network_latency();
let cpu_usage = Self::get_cpu_usage();
let memory_usage = Self::get_memory_usage();
// 동적 배치 크기 조정 알고리즘
```

```
match (network_latency, cpu_usage, memory_usage) {
(low, low, low) => 1000, // 높은 처리량
(medium, medium, medium) => 500, // 중간 처리량
_ => 100, // 안전한 처리량
}
}
}
```

2. 메모리 최적화

```
// 메모리 사용량 최적화
impl<T: Config> Pallet<T> {
fn optimize_memory_usage() {
// 오래된 데이터 정리
Self::cleanup_old_data();
// 캐시 크기 조정
Self::adjust_cache_size();
// 메모리 풀 관리
Self::manage_memory_pools();
}
fn cleanup_old_data() {
let current_block = <frame_system::Pallet<T>>::block_number();
let retention_period = T::DataRetentionPeriod::get();
let cutoff_block = current_block.saturating_sub(retention_period);
```

```
// 오래된 블록 데이터 제거
OldBlockData::<T>::remove_prefix(&cutoff_block, None);
// 오래된 트랜잭션 데이터 제거
OldTransactionData::<T>::remove_prefix(&cutoff_block, None);
}
}
```

3. 네트워크 최적화

```
// 네트워크 성능 최적화
pub struct NetworkOptimizer<T: Config> {
connection_pool: ConnectionPool,
bandwidth_monitor: BandwidthMonitor,
latency_tracker: LatencyTracker,
_phantom: PhantomData<T>,
}

impl<T: Config> NetworkOptimizer<T> {
pub fn optimize_connections(&mut self) {
// 연결 풀 최적화
self.connection_pool.optimize();
// 대역폭 사용량 모니터링
let bandwidth_usage = self.bandwidth_monitor.get_usage();
```

```
if bandwidth_usage > 0.8 {
self.reduce_message_frequency();
}
// 지연 시간 최적화
let avg_latency = self.latency_tracker.get_average();
if avg_latency > Duration::from_millis(100) {
self.optimize_routing();
}
}
fn reduce_message_frequency(&mut self) {
// 메시지 전송 빈도 감소
// 배치 처리 증가
// 압축 적용
}
fn optimize_routing(&mut self) {
// 최적 경로 선택
// 연결 품질 기반 라우팅
// 로드 밸런싱
}
}
```

4.3.13 장애 대응 및 복구

자동 복구 시스템

```rust
// 자동 장애 복구 시스템
pub struct FailureRecoverySystem<T: Config> {
health_checker: HealthChecker,
backup_manager: BackupManager,
alert_system: AlertSystem,
_phantom: PhantomData<T>,
}

impl<T: Config> FailureRecoverySystem<T> {
pub fn monitor_and_recover(&mut self) {
// 시스템 상태 확인
let health_status = self.health_checker.check_all();
match health_status {
HealthStatus::Critical(issue) => {
self.handle_critical_failure(issue);
},
HealthStatus::Warning(issue) => {
self.handle_warning(issue);
},
HealthStatus::Healthy => {
// 정상 상태
},
```

```
}
}
fn handle_critical_failure(&mut self, issue: CriticalIssue) {
// 즉시 알림 발송
self.alert_system.send_critical_alert(&issue);
match issue {
CriticalIssue::NodeUnresponsive => {
self.restart_node();
},
CriticalIssue::DatabaseCorruption => {
self.restore_from_backup();
},
CriticalIssue::NetworkPartition => {
self.handle_network_partition();
},
}
}
fn restore_from_backup(&mut self) {
// 최신 백업에서 복구
let latest_backup = self.backup_manager.get_latest_backup();
self.backup_manager.restore(latest_backup);
// 노드 재시작
self.restart_node();
```

```
}

}
```

백업 및 복구 전략

```
#!/bin/bash

# 파라체인 백업 스크립트


PARACHAIN_DATA_DIR="/var/lib/parachain"

BACKUP_DIR="/backup/parachain"

DATE=$(date  +%Y%m%d_%H%M%S)


# 데이터베이스 백업

echo  "Creating database backup..."

tar  -czf  "${BACKUP_DIR}/db_backup_${DATE}.tar.gz"  "${PARACHAIN_DATA_DIR}/chai


# 키스토어 백업

echo  "Creating keystore backup..."

tar  -czf  "${BACKUP_DIR}/keystore_backup_${DATE}.tar.gz"  "${PARACHAIN_DATA_DIR


# 체인 스펙 백업

echo  "Creating chain spec backup..."

cp  "${PARACHAIN_DATA_DIR}/chain_spec.json"  "${BACKUP_DIR}/chain_spec_${DATE}.j
```

```
# 오래된 백업 정리 (30일 이상)
find "${BACKUP_DIR}" -name "*.tar.gz" -mtime +30 -delete
find "${BACKUP_DIR}" -name "chain_spec_*.json" -mtime +30 -delete

echo "Backup completed: ${DATE}"
```

운영 베스트 프랙티스

4.3.14 보안 고려사항

1. 키 관리

```
// 안전한 키 관리 시스템
pub struct SecureKeyManager {
keystore: Keystore,
hsm_interface: Option<HsmInterface>, // Hardware Security Module
key_rotation_schedule: KeyRotationSchedule,
}

impl SecureKeyManager {
pub fn rotate_keys(&mut self) -> Result<(), KeyError> {
// 정기적인 키 로테이션
let current_keys = self.keystore.list_keys()?;
```

```
for key_type in [KeyType::Session, KeyType::Aura, KeyType::Grandpa] {
if self.should_rotate_key(key_type) {
let new_key = self.generate_new_key(key_type)?;
self.keystore.insert_key(key_type, new_key)?;
// 이전 키는 안전하게 보관 (복구용)
self.archive_old_key(key_type)?;
}
}
Ok(())
}
fn generate_new_key(&self, key_type: KeyType) -> Result<Key, KeyError> {
match &self.hsm_interface {
Some(hsm) => hsm.generate_key(key_type),
None => self.keystore.generate_key(key_type),
}
}
}
```

2. 네트워크 보안

```
// 네트워크 보안 설정
pub struct NetworkSecurity {
firewall_rules: Vec<FirewallRule>,
ddos_protection: DdosProtection,
```

```
rate_limiter: RateLimiter,
}

impl NetworkSecurity {
pub fn apply_security_policies(&self) {
// 방화벽 규칙 적용
for rule in &self.firewall_rules {
rule.apply();
}
// DDoS 보호 활성화
self.ddos_protection.enable();
// 속도 제한 설정
self.rate_limiter.set_limits(RateLimits {
rpc_requests_per_minute: 1000,
p2p_connections_max: 50,
xcm_messages_per_block: 100,
});
}
}
```

4.3.15 성능 모니터링 및 알림

모니터링 대시보드

```yaml
# monitoring-config.yml

monitoring:

metrics:

- name: "block_production_rate"

threshold: 0.5  # blocks per second

alert_condition: "below"

- name: "memory_usage"

threshold: 80  # percentage

alert_condition: "above"

- name: "disk_usage"

threshold: 85  # percentage

alert_condition: "above"

- name: "peer_count"

threshold: 10  # minimum peers

alert_condition: "below"



alerts:

- type: "email"

recipients: ["admin@parachain.com"]

- type: "slack"

webhook: "https://hooks.slack.com/..."

- type: "pagerduty"
```

```
service_key: "..."
```

자동화된 대응

```
// 자동화된 모니터링 및 대응 시스템
pub struct AutomatedResponse {
monitors: Vec<Box<dyn Monitor>>,
responders: Vec<Box<dyn Responder>>,
}

impl AutomatedResponse {
pub async fn run_monitoring_loop(&mut self) {
loop {
for monitor in &self.monitors {
if let Some(alert) = monitor.check().await {
self.handle_alert(alert).await;
}
}
tokio::time::sleep(Duration::from_secs(30)).await;
}
}
async fn handle_alert(&self, alert: Alert) {
for responder in &self.responders {
if responder.can_handle(&alert) {
```

```
responder.respond(&alert).await;

}

}

}

}
```

결론

파라체인 배포 및 운영은 기술적 복잡성과 운영상의 도전을 모두 포함하는 종합적인 과정입니다. 성공적인 파라체인 운영을 위해서는:

1. **체계적인 개발 및 테스트**: 로컬 환경에서 테스트넷까지 단계적 검증
2. **적절한 등록 방식 선택**: Agile Coretime vs 전통적 경매 시스템
3. **강력한 모니터링 시스템**: 실시간 성능 추적 및 자동 알림
4. **보안 중심 설계**: 키 관리, 네트워크 보안, 정기 감사
5. **자동화된 운영**: 장애 복구, 백업, 업그레이드 자동화
6. **지속적인 최적화**: 성능 튜닝 및 리소스 관리

이러한 모든 요소들이 조화롭게 작동할 때, 안정적이고 효율적인 파라체인 운영이 가능합니다.